select_functional_type_group_count.R

July 30, 2026

Contents

`select_functional_type_group_count`
Select Functional-Type Group Count

Description

Sweeps ‘functional_type_group_count_min‘ .. ‘functional_type_group_count_max‘, cuts the dendrogram at each value, and returns the group count that maximises the average silhouette width computed from ‘trait_dissimilarity‘.

The function also simulates the downstream pipeline filter chain for each candidate ‘k‘ and only considers values that leave at least ‘min_n_taxa‘ non-constant FT columns. The filter chain applied is: ‘filter_rare_taxa()‘, ‘filter_community_by_n_cores()‘, ‘filter_by_n_samples()‘, ‘prepare_community_for_fit()‘, optionally ‘compute_community_presence_absence()‘ (when ‘error_family‘ is ‘"binomial"‘), and ‘filter_constant_taxa()‘. The number of surviving columns is compared to ‘min_n_taxa‘. Among viable candidates the highest-silhouette ‘k‘ is returned. If no ‘k‘ is viable an error is raised via ‘cli::cli_abort()‘.

Usage

```
select_functional_type_group_count(
  trait_dissimilarity,
  hierarchical_clustering,
  functional_type_group_count_min = 10L,
  functional_type_group_count_max = 25L,
  data_community,
  minimal_proportion,
  min_n_taxa,
  min_n_cores,
  min_n_samples,
  error_family
)
```

Arguments

`trait_dissimilarity`
: A ‘dist‘ object produced by ‘compute_trait_dissimilarity()‘. Must inherit class ‘"dist"‘.

`hierarchical_clustering`
: An ‘hclust‘ object produced by ‘fit_hierarchical_clustering()‘. Must inherit class ‘"hclust"‘.

`functional_type_group_count_min`
: A single positive integer giving the minimum number of functional-type groups to evaluate. Must be at least 2 and no greater than ‘functional_type_group_count_max‘. Default: ‘10L‘.

`functional_type_group_count_max`
: A single positive integer giving the maximum number of functional-type groups to evaluate. Must be at least 2. If ‘functional_type_group_count_max‘ is greater than or equal to the number of observations, it is silently clamped to ‘n_observations - 1L‘. After clamping the maximum, ‘functional_type_group_count_min‘ is silently clamped to the adjusted maximum to handle datasets with very few taxa without erroring. Default: ‘25L‘.

`data_community`
: A long-format data frame with columns ‘taxon‘, ‘dataset_name‘, ‘age‘, and ‘value‘, as produced by ‘classify_to_functional_type()‘ or ‘classify_taxonomic_resolution()‘. Used to run the viability filter chain for each candidate ‘k‘.

`minimal_proportion`
: A single numeric value in (0, 1). A sample is considered to contain an FT group when the summed ‘value‘ for that group exceeds this threshold.

`min_n_taxa`
: A single positive integer. The minimum number of non-constant FT columns (i.e. groups present in strictly between 0 ‘minimal_proportion‘) that a candidate ‘k‘ must produce to be considered viable.

`min_n_cores`
: A single positive integer. Forwarded to ‘filter_community_by_n_cores()‘ during the viability check to remove FT groups present in fewer than this many distinct cores (‘dataset_name‘ values).

`min_n_samples`
: A single positive integer. Forwarded to ‘filter_by_n_samples()‘ during the viability check to remove FT groups present in fewer than this many distinct spatio-temporal samples (‘(dataset_name, age)‘ combinations).

`error_family`
: A single character string (e.g. ‘"binomial"‘). When ‘"binomial"‘, ‘compute_community_presence_ab is applied to the wide community matrix before ‘filter_constant_taxa()‘ during the viability check, mirroring the pipeline step.

Details

For each candidate group count in ‘functional_type_group_count_min‘.. ‘functional_type_group_count_max‘ the function calls ‘stats::cutree()‘ followed by ‘cluster::silhouette()‘ and records the mean silhouette width.

For each candidate ‘k‘ the community data are aggregated to the FT level (summed ‘value‘ per ‘(dataset_name, age, ft_group)‘) and then passed through the actual pipeline filter chain in sequence:

1. ‘filter_rare_taxa()‘ at ‘minimal_proportion‘.
2. ‘filter_community_by_n_cores()‘.
3. ‘filter_by_n_samples()‘.
4. ‘prepare_community_for_fit()‘ (wide matrix, using all surviving samples as their own IDs).

5. ‘compute_community_presence_absence()‘ if ‘error_family == "binomial"‘.
6. ‘filter_constant_taxa()‘.

The number of surviving columns is the viability count. If any step errors (e.g. all taxa removed), the count is treated as zero.

Selection proceeds as follows:

1. If at least one ‘k‘ has viability count >= ‘min_n_taxa‘: return the highest-silhouette among those ‘k‘ values.
2. If no ‘k‘ is viable: abort with ‘cli::cli_abort()‘.

Ties are broken by ‘base::which.max()‘ (first occurrence).

Value

A single integer giving the optimal number of functional-type groups (>= ‘functional_type_group_count_min‘ after clamping).

See Also

[compute_trait_dissimilarity()], [fit_hierarchical_clustering()], [assign_functional_type_clusters()]

Index